

Lambda Expressions Composition

CSC345: Programming Languages & Paradigms

Motivation

“Sum of squares of positive integers” function:

```
f :: [Int] -> Int
f xs = foldr (+) 0 (map sqr (filter pos xs))
  where
    sqr x = x * x
    pos x = x > 0
```

cannot simplify to:

```
f xs = foldr (+) 0 (map (x * x) (filter (x > 0) xs))
```

Error! x not in scope

- Need some way of assuming that **x** is the argument

More Motivation

- We're currently able to define functions, but so far we *always* have to give functions a name.

```
addOne :: Int -> Int  
addOne x = x + 1
```

Nameless Functions

Defining a nameless function in Haskell:

$\backslash x \rightarrow x + 1$

- “the function that takes x and returns the value of 1 added to it”
- “assuming x is the argument, return $x + 1$ ”
- $\backslash$ signals that it is a **lambda function**
 - $\backslash$ symbol on the keyboard looks similar to the lambda character λ
 - Lambda calculus (Alonzo Church 1903-1955) – theory of functions
 - Lambda expressions came to Java in 2014, 55 years after they came to functional programming

Example

```
f :: [Int] -> Int
f xs = foldr (+) 0 (map sqr (filter pos xs))
  where
    sqr x = x * x
    pos x = x > 0
```

“Sum of squares of positive integers” function:

```
f xs = foldr (+) 0 (map (\x -> x * x) (filter (\x -> x > 0) xs))
```

- x is arbitrary, can use any identifier

Sections

Convenient shorthand for writing partially-applied functions:

- “**x** goes in place of missing argument”

General Form:

(x #) = \y -> x # y

(# y) = \x -> x # y

(#) = \x -> (\y -> x # y)

In Practice:

(> 0) is shorthand for (\x -> x > 0)

(2 *) is shorthand for (\x -> 2 * x)

(+ 1) is shorthand for (\x -> x + 1)

(2 ^) is shorthand for (\x -> 2 ^ x)

(^ 2) is shorthand for (\x -> x ^ 2)

Example

“Sum of squares of positive integers” function:

```
f xs = foldr (+) 0 (map (\x -> x * x) (filter (\x -> x > 0) xs))
```

- x is arbitrary, can use any identifier

... now becomes ...

```
f xs = foldr (+) 0 (map (^ 2) (filter (> 0) xs))
```

More Examples

```
> map (\x -> x + 1) [2,3,4]  
[3,4,5]
```

```
> (\x -> x + x) 2  
4
```

```
add :: Int -> Int -> Int  
add x y = x + y
```

can be understood as meaning:

```
add :: Int -> (Int -> Int)  
add = \x -> (\y -> x + y)
```

```
const :: a -> b -> a  
const x _ = x
```

can be understood as meaning:

```
const :: a -> (b -> a)  
const x = \_ -> x  
or  
const = \x -> (\_ -> x)
```

Function that returns the first n odd integers:

```
odds :: Int -> [Int]  
odds n = map (\x -> 2*x+1) [0..n-1]
```


Function Composition

- Function composition operator that is already defined in Haskell:
 - “the dot operator” `.`
 - Passes the output of one function to another

Function Signature:

$(.) :: (b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow (a \rightarrow c)$

Function Definition:

$(f . g) x = f (g x)$

- Takes two functions, and produces a function that performs one function, and then the other function
- **Prompt:** why is the type signature not: $(a \rightarrow b) \rightarrow (b \rightarrow c) \rightarrow (a \rightarrow c)$

Composition Example

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)  
(f . g) x = f (g x)
```

(We could also do this example with lambdas, but forgetting about nameless functions for a minute...)

```
sqr :: Int -> Int  
sqr x = x * x
```

```
pos :: Int -> Bool  
pos x = x > 0
```

```
> (pos . sqr) 3  
pos (sqr 3)  
pos 9  
True
```

Compare and Contrast:

```
possqr :: Int -> Bool  
possqr x = pos (sqr x)  
> possqr 3  
pos (sqr 3)  
pos 9  
True
```

```
possqr :: Int -> Bool  
possqr = pos . sqr  
> possqr 3  
(pos . sqr) 3  
pos (sqr 3)  
pos 9  
True
```

Composition is Associative

$$(f . g) . h = f . (g . h)$$

- Always apply the “second / rightmost” function first

$$f . g . h$$

- No parentheses, not ambiguous
- first apply h , then g , finally f

Example

“Sum of squares of positive integers” function:

```
f xs = foldr (+) 0 (map (^ 2) (filter (> 0) xs))
```

... now becomes ...

```
f = foldr (+) 0 . map (^ 2) . filter (> 0)
```

- `f` still has type `[Int] -> Int`
- don't have to think about argument `xs` anymore
- don't need parentheses because composition is associative

Example Trace:

```
> f [1,2,(-2)]  
(foldr (+) 0 . map (^ 2) . filter (> 0)) [1,2,(-2)]  
foldr (+) 0 (map (^ 2) (filter (> 0) [1,2,(-2)]))  
foldr (+) 0 (map (^ 2) [1,2])  
foldr (+) 0 [1,4]  
5
```

More Examples

odd $n = \text{not } (\text{even } n)$

... to ...

odd = not . even

twice $f\ x = f\ (f\ x)$

... to ...

twice $f = f . f$